

Task 1: Fine-tune LLM for Language Translation

Hoang Huy Thai - 428955

RPTU Kaiserslautern, Department of Computer Science

Note: This report contains a project documentation and reflection on the portfolio task submitted for the lecture Engineering with Generative AI in WiSe 2024-25. This report is an original work and will be scrutinised for plagiarism and potential LLM use.

1 Portfolio documentation

1.1 Research Phase

1.1.1 WMT24++

WMT24++ [1] is a multilingual dataset comprising 55 languages, including 2 pairs of en-de and en-fr. Although there is no direct mapping from German to French, each pair in the dataset shares the same English source. This still ensures the accuracy of the dataset in terms of the translation task from German to French. The dataset contains 998 data points that cover 4 distinct domains: literary, news, social, and speech. Table 1 shows the statistics on the WMT24 dataset for English sources in all domains.

Although the size of the dataset does not strictly meet the task's requirement (at least 1000

Table 1: Domain coverage and text length statistics for the WMT24 dataset [1].

Domain	Para	Token/Para	Doc	Token/Doc
Canary	1	-	1	-
Literary	206	38.1	8	980.0
News	149	54.1	17	473.8
Social	531	15.7	34	245.0
Speech	111	73.0	111	73.0
All	998	32.4	171	189.0

pairs), the quality of the dataset outclasses other components due to the human-written references and post-edits for the data. Figure 1 shows us that the quality of French and German human translations are improved by post-editing.

1.1.2 Llama 3.2 1B Instruct

LLama 3.2 1B Instruct is the lightweight version of Llama 3 family with 1.23B of parameters [2], which sizes at bfloat16 (BF16) precision. Also with the support of quantization, the pretrained model is claimed to perform faster, safer and more compact than the counterparts, but experiences same accuracy as the non-quantized models [3]. Therefore, Llama-3.2-1B-Instruct is

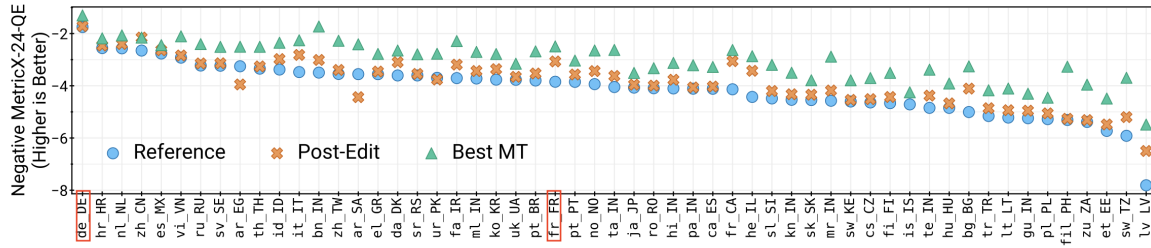


Figure 1: A comprehensive comparison of the quality of the reference, the post-edit, and highest quality MT system, evaluated by MetricX-24-QE [1].

possibly fine-tuned with Colab notebook (T4 GPU, 15GB VRAM, 12.7 GB RAM) without any compromises of model’s performance.

Beside the capabilities of code execution and synthetic data generation, Llama 3 also offers a more powerful multilingual capability especially in German and French by using expert training and multilingual data collection - SFT Data collected from human annotations, data from other NLP tasks, rejection sampled data and translated reasoning data [4].

In the context of only 1000 pairs of German-French used as dataset, the instruct model is more efficient than the base model because it is already tailored to follow the instructions if we define them properly. Meanwhile, the base model might need more data to learn the instructions to achieve a decent performance as its counterpart.

1.2 Design Phase

1.2.1 Fine-tuning approach

QLoRA (Quantized LoRA), a quantized version of LoRA, quantizes a pretrained model to 4-bit and makes it more efficient to fine-tune with a limited of available hardware by reducing average memory requirements of finetuning without degrading the runtime and the model’s performance in comparison to original baseline [5].

Along with LoRA, a lightweight fine-tuning approach that only requires updating a small number of parameters, which makes the fine-tuning process on Collab faster and memory-efficient. The fine-tuned models are also lightweight, which are more feasible to store and use in practice (only 99 MB in my case).

1.2.2 Split ratio of dataset

Because of the high complexity, this project is focused only on training and testing. This means that the validation step to find the best set of hyperparameters is left for future improvement and dataset is split according to the ratio of 80-20 (a common ratio): 80% of dataset for training and 20% of dataset for testing.

In the context of WMT24++, there are 800 pairs of German-French used for fine-tuning and others (198 pairs) used for testing.

1.2.3 Synthetic data

In this task, we are required to generate a synthetic dataset with twice the size of the training set above. The used LLM also needs to meet the size requirement of at least 32B parameters and requirement of public usage. Due to the given constraints, the model Qwen/Qwen2.5-Coder-32B-Instruct is used to generate the synthetic dataset for the translation of German-French. From the experiments, we observe that it is impossible to obtain the dataset with the required size in only one API call because:

- The response is limited by maximum token. In terms of Qwen/Qwen2.5-Coder-32B-Instruct, the sum of the input tokens and the generated tokens must less than or equal to 4096 (context window size).
- In spite of explicitly mentioning about the uniqueness of each pair in the dataset, there are a large amount of repeated pairs.

Therefore, we will break down the dataset into smaller batches and for each batch, a distinct prompt will be used to guarantee the uniqueness of dataset. Also, the number of pairs will be reduced to ensure that the model does not repeat itself. In my case, the ideal threshold is 20 pairs. Moreover, to ease the preprocessing of response for exceeding steps, the criteria for structure and format of response are also defined in prompt.

First of all, we need to generate a large number of distinct prompts for API calls. To achieve this, the initial idea is that we will parameterize the prompt with different parameters. One of them are topics of the pairs, which must be well covered by Qwen/Qwen2.5-Coder-32B-Instruct. To obtain a list of topics, the following prompt is used to ask Qwen/Qwen2.5-Coder-32B-Instruct itself to generate the most useful topics:

You are an intelligent assistant that is bilingual in German and French.

Your task is to:

- *List out 21 topics that well-covered by you and also available in both German and French.*
- *Results must be in English.*
- *Do not include any explanations.*
- *Return the response strictly in list format and the topics are separated by new line.*

After that, we will generate all the possible prompts by permutating over all the parameters: topic of translation, length of the sentence, complexity and style.

You are a helpful assistant that generates synthetic datasets. Your task is:

- *Generate 20 pairwise German-French translations on the topic of {} with {} length and {} complexity.*
- *Remove the duplicate pairs in the response.*
- *Return the response strictly in CSV format, like this:*

German||French

Heute ist der Himmel blau.||Le ciel est bleu aujourd'hui.

Es regnet leicht.||Il pleut lgrement.

- *Do not include any explanations. Only return the CSV data.*

This prompt format will make sure that the response is well-structured in CSV format and not include any repetitive pairs.

1.2.4 Metrics

BLEU Bilingual evaluation understudy (BLEU), a lexical-based metric, is used to measure the similarity of the hypothesis to the reference by comparing n-grams of prediction with the n-grams of reference translation and counting the number of matches [6]. Due to its simplicity and fastness, BLEU score is widely used to evaluate machine translation and text generation models. This is also the main reason why BLEU score is used in this project is to compare the model's performance with other translation models.

However, BLEU score has some drawbacks. It only considers exact word matching and ignores the semantic meaning like synonyms, which worsens the likelihood of exact n-gram matches if the sentences are lengthy [7].

METEOR The metric for evaluation of translation with explicit ordering (METEOR), a lexical-based metric, is designed to solve the problems of BLEU score predicated on idea of unigram matching between the prediction and reference translation [8]. In addition to exact word matching, METEOR also supports word matching between morphological variants or synonyms of each other. Although METEOR is generally better than BLEU score for semantic capture, it still encounters some problems with long sentences especially paraphrasing and reordering cases.

BERTScore BERTScore, a contextual embedding metric, uses contextual embeddings from BERT models to measure the similarity between the prediction and reference translation [7]. It addresses two common problems in n-gram-based metrics like BLEU or METEOR: fail to robustly match paraphrases and handle word order changes [9]. However, BERTScore does not always align with human references, which BLEU and METEOR excel at.

Three metrics have their own pros and cons. To be objective in model evaluation, all of them are used to measure the performance's translation models.

1.3 Implementation Phase

The implementation of the project is grouped into 13 groups in according to 13 sections in Jupyter Notebook. This part will briefly describe each sector and its purpose.

1.3.1 Environment Setup

The goal is to:

- Prepare necessary packages for the project.
- Set up hardware and huggingface account.
- Define global variables.

1.3.2 Linguistic Dataset

Match dataset Because there is no direct mapping from German to French, we need to load 2 datasets en-fr and en-de; and then map from de to fr to generate pairwise German-French dataset for fine-tuning.

Format dataset To optimize the fine-tuning on LLama 3 model, we will transform pairwise dataset into instruction prompt following model card [10] suggested by Meta. The prompt consists of 3 main parts:

- System: instruction for model.
- User: the request from user.
- Assistant: the response are expected to receive from the model.

This is an example of one instruction data point after transformation:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
```

```
You are an assistant that translates German to French. You only reply in French and only give the answer without any explanations.<|eot_id|>
```

```
<|start_header_id|>user<|end_header_id|>
```

```
Sisos Darstellungen von Land und Wasser in neuer Ausstellung<|eot_id|>
```

```
<|start_header_id|>assistant<|end_header_id|>
```

```
Exposition des travaux de Siso sur la terre et l'eau, la nouvelle galerie du centre<|eot_id|>
```

1.3.3 Metrics

The goal is to:

- Implement 3 metrics: BLEU, METEOR and BERTScore.
- Implement inference function for model and extraction function.

1.3.4 Fine-tune Approach

The goal is to:

- Load quantized model.
- Define LoRa configurations.
- Define training configurations.

1.3.5 Fine-tuning

Section 6, 9 and 11 focus on fine-tuning the large language model on different datasets with configurations defined in 1.3.4.

1.3.6 Evaluation

Section 5, 7, 10 and 12 is focus on evaluating the performance of model A, B, C, D in 1.3.5 by metrics in 1.3.3.

1.4 Results

The figure 2 and the table 2 demonstrate that model B outperforms the others in all the metrics. All the models perform worst on BLEU while they have higher scores in BERTScore and METEOR, which can conclude that they are good at capturing the meaning of translation rather than exact word matching.

Regarding to model A, it has the worst performance across all metrics with extremely low BLEU score of 0.01. Its BERTScore (0.6540) and Meteor (0.1230) are also the lowest but more decent than BLEU, indicating that model still retains some meaning in spite of the low lexical similarity.

Model B ranks at the first place with the dominance of 3 metrics (BLEU: 0.1171, BERT: 0.7936, METEOR: 0.3763). It witness a significant improvement in performance after fine-tuning Llama model on WMT24++ even though the BLEU score is still on the low level. The lengthy sentences and small number of training dataset could be the reason why the fine-tuned model produces a poor performance on BLEU.

As regards to model C, its scores are increased across all metrics, but still worse than model A as expected. The difference between WMT24++ and synthetic dataset generated by Qwen/Qwen2.5-Coder-32B-Instruct and the bias of test dataset could be the primary causes. While the data in WMT24++ is of great length and high complexity, synthetic data is shorter and simpler.

The above reasons are also the reason why model D produces slightly poorer performance than model B, which is expected to be best model due to larger amount of dataset (WMT24++ and synthetic dataset).

The performance of 4 translation models

Model B tends to be the best among them.

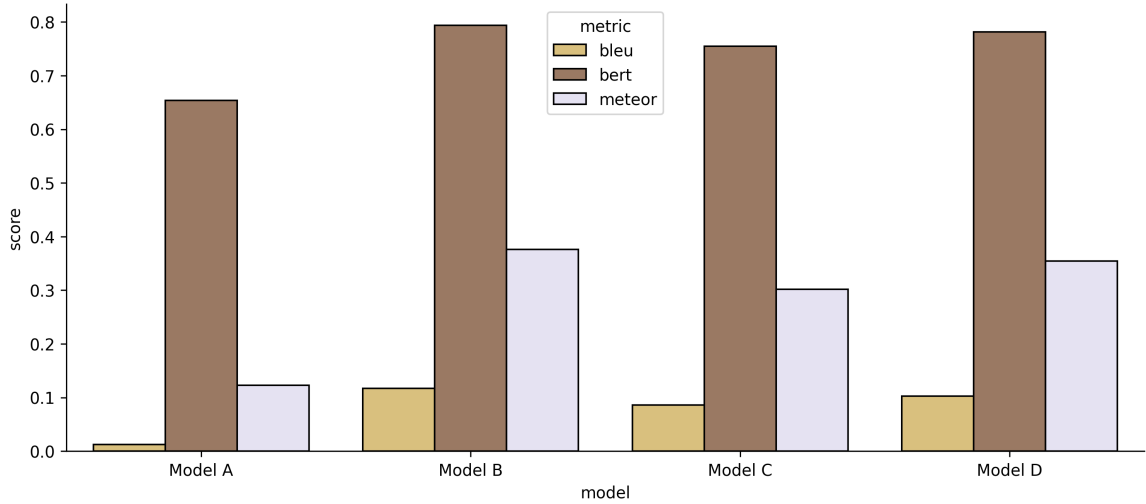


Figure 2: A illustration of the performance of 4 models.

Table 2: A evaluation result of 4 models using BLEU, BERTScore and METEOR.

Model	BLEU	BERTScore	METEOR
Model A	0.0124	0.6540	0.1230
Model B	0.1171	0.7936	0.3763
Model C	0.0857	0.7550	0.3020
Model D	0.1027	0.7817	0.3545

2 Reflection

1. What was the most interesting thing that you learnt while working on the portfolio? What aspects did you find interesting or surprising?

Answer: The most exciting part of the portfolio exam was the development of an end-to-end approach to fine-tuning LLMs with reduced effort and minimal resources:

- In the past, training a model for a specific task like translation required building the model from scratch, which was an extravagant process in terms of effort, cost, and resources. However, with the advent of lightweight fine-tuning approaches such as QLoRA or Unsloth, we can now simplify the process by leveraging general large language models.
 - Prior to my current role, I worked as a collaborator in the Quality Control Department of an AI company. This experience taught me that constructing training datasets for machine learning models is highly expensive and time-consuming, particularly for small and medium-sized companies. Therefore, the emergence of large language models specialized in synthetic data generation, such as llama-3.1-405b-instruct, offers a more affordable approach to constructing datasets.
 - Ready-to-use API services provided by HuggingFace are of great importance in bridging the gap between research and application of machine learning models. With the advent of limited-resource devices like smartphones, we can now integrate generative AI models through API calls, making this approach more feasible than building our own models and saving them locally.
 - Thank to libraries such as HuggingFace and Unsloth, the process of fine-tuning models using Python has become increasingly accessible to a broader audience, especially beginners who may lack a comprehensive understanding of the underlying mechanisms of LLMs.
2. Which part of the portfolio are you proud of? Why? What were the challenges you faced, and how did you overcome them?

Answer: I am most proud of research phase which is also the most important phase of task one. Pre-trained model, dataset and fine-tuning approach play important roles in role as backbone of the project. Due to the constraints of resources, dataset size and task requirements, it requires a thorough research and comparison among the available options. The compatibility among the components is also considered to make sure that

all of them work seamlessly with each other. Therefore, at this early phase, researching and prototyping workflow have to be done simultaneously to ensure that the final implementation satisfies the requirements and models reach to the peak performance. Being lost and overwhelmed is the biggest obstacle I have dealt with during the phase. Each of model, dataset or fine-tuning approach has its own pros and cons. Completing a literature review and deep analysis on characteristics of models, datasets and fine-tuning approach and make decisions based on clear predefined constraints are the key step leading to the success of this step.

3. What adjustments to your design and implementation were necessary during the implementation phase? What would you change or do differently if you had to do the portfolio task a second time? What would be potential areas for future improvement?

Answer: During the implementation phase, there are some expected problems occurring, which imposes some modifications on the the design and implementation.

- One of the most common problems I have experienced is the CUDA out of memory during fine-tuning model or inferencing model, which indicates that the current process already exceeds maximum of available resources. Therefore, we need to optimize the fine-tuning process so that it can fit within hardware. Here is a list of feasible solutions to the error: try more lightweight model, quantize model to 4-bit or modify the training parameters like `per_device_train_batch_size`.
- The fine-tuned model does not know how to stop generating text or generate random text after fine-tuning on translation dataset. Wrong data format might be the root cause of the problem so we need to change the format of training dataset, which may already be defined by the providers.
- Due to the strict constraint of dataset size, some large language models like MT5 can not learn the translation task efficiently, which leads to poor performance on the test dataset. At this time, we need to change our model, which could make a compromise among the given constraints.

If I were given a second chance, I would have chosen another approach to generate synthetic dataset for fine-tuning model on translation task. As mentioned in 1.4, there are some conflicts between WMT24++ and synthetic dataset in terms of length and complexity leading to poor scores on the final model.

Potential aspects of the project we could consider for the future improvement is:

- Increase the number of training dataset: we can fine-tune our model on a larger translation dataset like WMT19 or generate more synthetic dataset by more powerful large language models, which is consistent with the original dataset.
- Hyperparameter Tuning: we can split the dataset into 3 smaller sets (train, validation and test); and the validation set will be utilized to find the set of hyperparameters that produces best performance on it.
- Test the pretrained model with state-of-the-art fine-tuning approaches, which is implemented by PyTorch or Tensorflow and not ready-to-use like HuggingFace libraries.
- Conduct a profound reseach on another LLMs that is more specialized in synthetic data generation than Qwen/Qwen2.5-Coder-32B-Instruct, which is well-known for code generation.

4. Include a brief section on ethical considerations when using these models on language translation tasks.

Answer: When using machine learning models, consider these ethical matters:

- Privacy: Ensure personal information isn't saved in the system when translating.
- Bias: Fine-tuning models on hate speech or misleading information can harm the community, culture sensitivity, and minorities.
- Accuracy: Mistranslations in critical fields like medicine and law can have serious consequences.

5. From the lecture/course including guest lectures, what topic excited you the most? Why? What would you like to learn more about and why?

Answer: From the Generative AI course, I was impressed by the Amazon lecture on AI/ML innovations. It briefly explains how AI/ML is efficiently applied in business. The lecture is accessible and focuses on Amazon's approach to use generative AI. It starts with customer needs, designs efficient systems, and discusses risks, opportunities, and challenges when applying generative AI in practice, offering a unique business perspective.

I would like to learn more about the infrastructure of the AI system because it is the core of the system to ensure that every component works seamlessly with others. Also, I am curious about how generative AI is applied in business and how to solve end-to-end problems from the viewpoint of business.

6. How did you find working with DIFY platform during the course work? Would you recommend using DIFY in learning Generative AI technologies and why? What is the best start for learning Generative AI either by Python code or No-code platforms and why?

Answer:

Working with the DIFY platform during the coursework offers me practical experience and a general overview of generative AI applications' workflow. The DIFY platform comes in handy in case we desire to find an efficient option to apply large language models into business with little cost. Also, the platform is a great starting point for beginners or non-technical staff who have a desire to scrutinize what is inside the box because it provides us with an interactive interface and user-friendly design. Additionally, with the accessibility of documentations on the DIFY website, the users can easily learn how to design a suitable system that meets their requirements.

From my own experience, the no-code platform like DIFY is a good kick-off of the Generative AI journey because jumping into the code immediately would degrade the motivation of beginners who might not have any knowledge about coding in advance or even if they have, it is still a better choice to provide them with a panoramic view about Generative AI.

7. How did you find the assignments and exercises in the course and how they help you in the portfolio exams?

Answer: The assignments and exercises are very useful and informative. They apply the knowledge we learn in lectures to practice. The exercises focus on using models and

applying approaches correctly and efficiently, rather than the underlying algorithms. Each exercise is like a puzzle that we combine to solve the portfolio exam. For task 1, the exercises helped me use the HuggingFace libraries and develop my ideas based on the provided skeletons.

References

- [1] Daniel Deutsch, Eleftheria Briakou, Isaac Caswell, Mara Finkelstein, Rebecca Galor, Juraj Juraska, Geza Kovacs, Alison Lui, Ricardo Rei, Jason Riesa, et al. Wmt24++: Expanding the language coverage of wmt24 to 55 languages & dialects. *arXiv preprint arXiv:2502.12404*, 2025.
- [2] HuggingFace. Hugging face llama 3.2 website, 2025. Accessed: 10-Mar-2025.
- [3] Meta. Llama 3.2 website, 2025. Accessed: 10-Mar-2025.
- [4] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems*, 36:10088–10115, 2023.
- [6] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- [7] Seungjun Lee, Jungseob Lee, Hyeonseok Moon, Chanjun Park, Jaehyung Seo, Sugyeong Eo, Seonmin Koo, and Heuseok Lim. A survey on evaluation metrics for machine translation. *Mathematics*, 11(4):1006, 2023.
- [8] Satanjeev Banerjee and Alon Lavie. Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. In *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization*, pages 65–72, 2005.
- [9] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*, 2019.
- [10] Meta. Model cards prompt formats, 2025. Accessed: 11-Mar-2025.